CS/CE 224/272 - Object Oriented Programming and Design Methodology

Nadia Nasir

Habib University, Fall 2025

Multidimensional Arrays on the Stack (Compile-Time)

Multidimensional Arrays

```
1 | int a[3][5]; // a 3-element array of 5-element arrays of int
```

With a two-dimensional array, it is convenient to think of the first (left) subscript as selecting the row, and the second (right) subscript as selecting the column. Conceptually, we can imagine this two-dimensional array laid out as follows:

```
// col 0     col 1     col 2     col 3     col 4
// a[0][0]   a[0][1]   a[0][2]   a[0][3]   a[0][4]   row 0
// a[1][0]   a[1][1]   a[1][2]   a[1][3]   a[1][4]   row 1
// a[2][0]   a[2][1]   a[2][2]   a[2][3]   a[2][4]   row 2
```

Multidimensional Arrays

To access the elements of a two-dimensional array, we simply use two subscripts:

```
1 | a[2][3] = 7; // a[row][col], where row = 2 and col = 3
```

C++ even supports multidimensional arrays with more than 2 dimensions:

```
1 | int threedee[4][4][4]; // a 4x4x4 array (an array of 4 arrays of 4 arrays of 4 ints)
```

Multidimensional Arrays

- We mentioned that 1D arrays are laid out sequentially in memory (contiguous elements).

- For 2D arrays, the elements are arranged in a **row-major** order.

```
[0][0] [0][1] [0][2] [0][3] [0][4] [1][0] [1][1] [1][2] [1][3] [1][4] [2][0] [2][1] [2][2] [2][3] [2][4]
```

- Good to know because when accessing them, your **outer loop can iterate over rows**, and **inner loop can iterate over columns**.

Multidimensional Arrays

```
1  int array[3][5]
2  {
3    { 1, 2, 3, 4, 5 },      // row 0
4    { 6, 7, 8, 9, 10 },     // row 1
5    { 11, 12, 13, 14, 15 } // row 2
6  };
```

Just like normal arrays, multidimensional arrays can still be initialized to 0 as follows:

```
1  int array[3][5] {};
```

An initialized multidimensional array can omit (only) the leftmost length specification:

```
int array[][5]
{
  { 1, 2, 3, 4, 5 },
  { 6, 7, 8, 9, 10 },
  { 11, 12, 13, 14, 15 }
};
```

In such cases, the compiler can do the math to figure out what the leftmost length is from the number of initializers.

Omitting non-leftmost dimensions is not allowed:

```
int array[][]
{
  { 1, 2, 3, 4 },
  { 5, 6, 7, 8 }
};
```

Multidimensional Arrays

```cpp
#include <iostream>

int main()
{
    int arr[3][4]
    {
        { 1, 2, 3, 4 },
        { 5, 6, 7, 8 },
        { 9, 10, 11, 12 }
    };

    std::cout << sizeof(arr) / sizeof(arr[0]) << std::endl;          // 3
    std::cout << sizeof(arr[0]) / sizeof(arr[0][0]) << std::endl;    // 4

    return 0;
}
```

Multidimensional Arrays

If you want to pass a 2D array to a function,

```
void foo(int arr2D[][5], int rows);     // 5 is the number of columns in the 2D array
```

Multidimensional Arrays on the Heap (Runtime)

You can now get varying number of columns in each row.

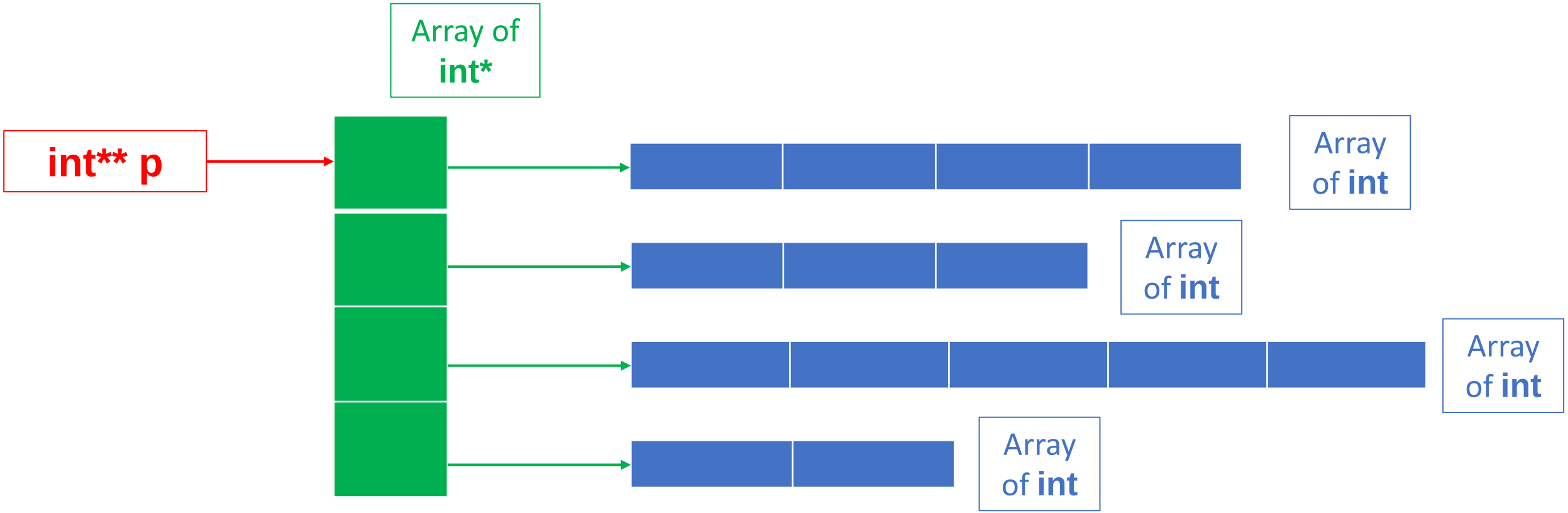
This wasn’t possible with the nD-arrays on the stack.

Varying Length of Rows

- Creating 2D-arrays over the stack means that row lengths would always be constant.

- If you need varying row lengths, you can allocate memory on the heap.

```
int** p{ new int*[4] };        // this creates the array of int*
p[i] = new int[len];           // this creates the ith array of int
```

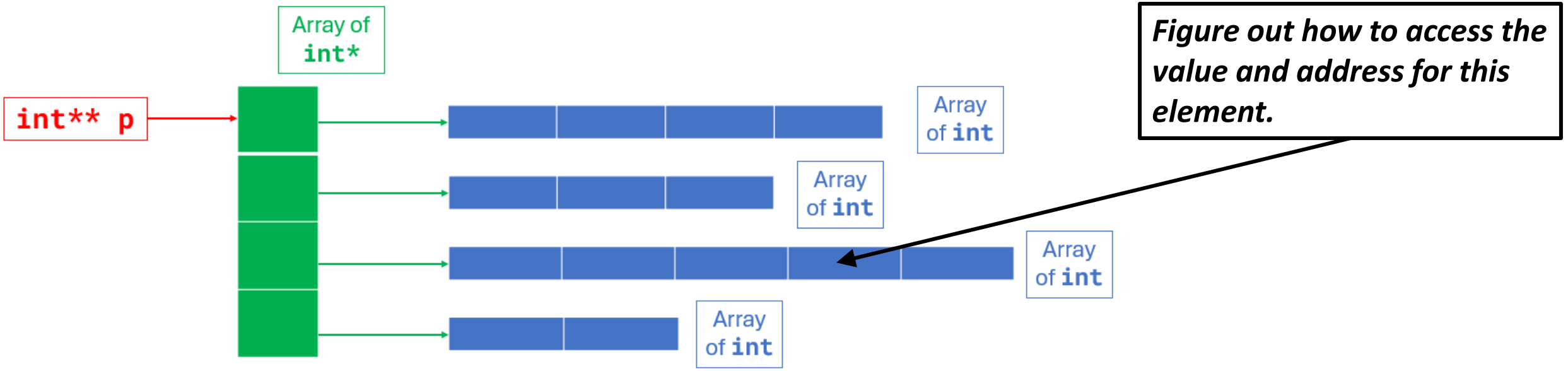


Varying Length of Rows

- Note that you **first** have to allocate the array of **int***
- Each element of this array can have an integer type address as its value.
- So against each element of this **int*** array, we dynamically allocate an array of **int**
- Recall that **p[i]** is identical to ***(p + i)**
  - Both of these will give you access to the data or the value stored at index **i** in the **int*** array.

```cpp
#include<iostream>

int main()
{
    int** p{ new int*[4]{} };

    p[0] = new int[4]{1, 2, 3, 4};
    p[1] = new int[3]{5, 6, 7};
    p[2] = new int[5]{8, 9, 10, 11, 12};
    p[3] = new int[2]{13, 14};

    std::cout << "p[2][3] = "           << p[2][3]           << '\n';
    std::cout << "*(p[2] + 3) = "       << *(p[2] + 3)       << '\n';
    std::cout << "*( *(p + 2) + 3 ) = " << *( *(p + 2) + 3 ) << '\n';
    std::cout << "( *(p + 2) )[3] = "   << ( *(p + 2) )[3]   << '\n';

    for(int i{}; i<4; i++)
    {
        delete[] p[i];
    }

    delete[] p;

    return 0;
}
```

```
p[2][3] = 11
*(p[2] + 3) = 11
*( *(p + 2) + 3 ) = 11
( *(p + 2) )[3] = 11
```



In order to **access each element**, you can treat this **like a 2D array**.

So to access the marked element, you first need to **go to the correct row**.
We can do that by, **p[2]** or ***(p + 2)** in this example.

Once we're in the correct row, we need to **get to the correct element (or correct column)**. Do that by,

- **p[2][3]** /* easy to read; other ways are shown so that you understand what's going on, and what can be done. */
- ***(p[2] + 3)**
- ***(*(p + 2) + 3)**
- **(*(p + 2))[3]**

```cpp
#include<iostream>

int main()
{
    int** p{ new int*[4]{} };

    p[0] = new int[4]{1, 2, 3, 4};
    p[1] = new int[3]{5, 6, 7};
    p[2] = new int[5]{8, 9, 10, 11, 12};
    p[3] = new int[2]{13, 14};

    std::cout << "p[2][3] = "           << p[2][3]           << '\n';
    std::cout << "*(p[2] + 3) = "       << *(p[2] + 3)       << '\n';
    std::cout << "*( *(p + 2) + 3 ) = " << *( *(p + 2) + 3 ) << '\n';
    std::cout << "( *(p + 2) )[3] = "   << ( *(p + 2) )[3]   << '\n';

    for(int i{}; i<4; i++)
    {
        delete[] p[i];
    }

    delete[] p;

    return 0;
}
```

```
p[2][3] = 11
*(p[2] + 3) = 11
*( *(p + 2) + 3 ) = 11
( *(p + 2) )[3] = 11
```

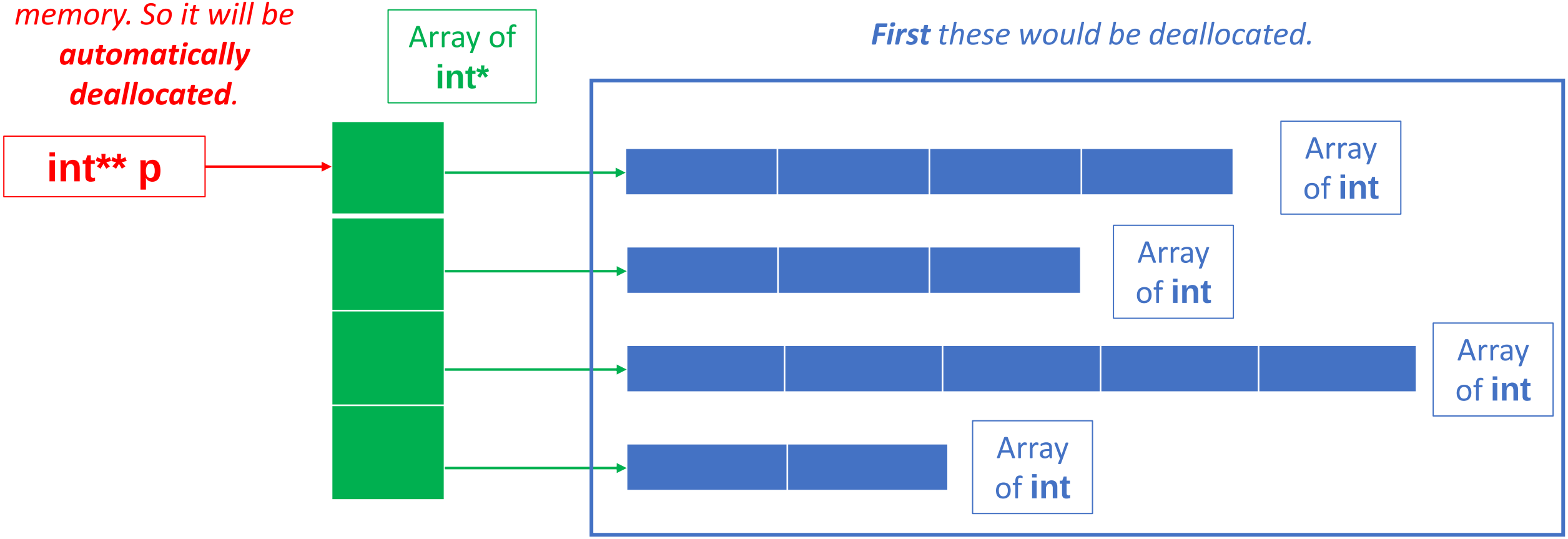
Varying Length of Rows

- All of the memory that you have taken from the heap **must be manually deallocated using delete**.

Note that this is already on the stack memory. So it will be ***automatically deallocated****.*

Then *deallocate this pointer type array.*

First *these would be deallocated.*



```cpp
#include<iostream>

int main()
{
    int** p{ new int*[4]{} };

    p[0] = new int[4]{1, 2, 3, 4};
    p[1] = new int[3]{5, 6, 7};
    p[2] = new int[5]{8, 9, 10, 11, 12};
    p[3] = new int[2]{13, 14};

    std::cout << "p[2][3] = "           << p[2][3]           << '\n';
    std::cout << "*(p[2] + 3) = "       << *(p[2] + 3)       << '\n';
    std::cout << "*( *(p + 2) + 3 ) = " << *( *(p + 2) + 3 ) << '\n';
    std::cout << "( *(p + 2) )[3] = "   << ( *(p + 2) )[3]   << '\n';

    for(int i{}; i<4; i++)
    {
        delete[] p[i];
    }

    delete[] p;

    return 0;
}
```

```
p[2][3] = 11
*(p[2] + 3) = 11
*( *(p + 2) + 3 ) = 11
( *(p + 2) )[3] = 11
```

Varying Length of Rows

- If you deallocate the **int*** array first, you will create a memory leak of all the **int** arrays.

Passing 2D array on the heap to a function

```
void foo(int** arr2D);
```

You would also need to pass the number of rows.

For the number of columns, it can depend on the situation; if you know that you are working with a jagged array (each row can have a different length), you could pass an array that contains all the column lengths.

Static Variables

- Code file.